

Unit 4

Statistical Reasoning

CONTENTS

- **Probability & Bayes' theorem**
- **Bayesian networks**
- **Certainty factors & rule-based systems**
- **Weak slot and filler structures**
- **Semantic nets**
- **Frames Strong slot and filler structures**
- **Conceptual dependency**

Uncertainty

- The knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates.
- With this knowledge representation, we might write $A \rightarrow B$, which means if A is true then B is true,
- but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called uncertainty.
- So to represent uncertain knowledge, where we are not sure about the predicates, we need uncertain reasoning or probabilistic reasoning.

Causes of uncertainty

Following are some leading causes of uncertainty to occur in the real world.

- 1.Information occurred from unreliable sources.
- 2.Experimental Errors
- 3.Equipment fault
- 4.Temperature variation
- 5.Climate change.

Probabilistic reasoning

- Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge.
- In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.
- We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.
- In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "It will rain today," "behavior of someone for some situations," "A match between two teams or two players."
- These are probable sentences for which we can assume that it will happen but not sure about it, so here we use probabilistic reasoning.

Need of probabilistic reasoning in AI:

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

Bayes' rule

Bayesian Statistics

□ **Probability:** Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.

$0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A .

$P(A) = 0$, indicates total uncertainty in an event A .

$P(A) = 1$, indicates total certainty in an event A .

We can find the probability of an uncertain event by using the below formula.

$$\text{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- $P(\neg A)$ = probability of a not happening event.
- $P(\neg A) + P(A) = 1$.

□ **Event:** Each possible outcome of a variable is called an event.

□ **Sample space:** The collection of all possible events is called sample space.

□ **Random variables:** Random variables are used to represent the events and objects in the real world.

□ **Prior probability:** The prior probability of an event is probability computed before observing new information.

□ **Posterior Probability:** The probability that is calculated after all evidence or information has taken into account. It is a combination of prior probability and new information.

Conditional probability:

Conditional probability is a probability of occurring an event when another event has already happened.

Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

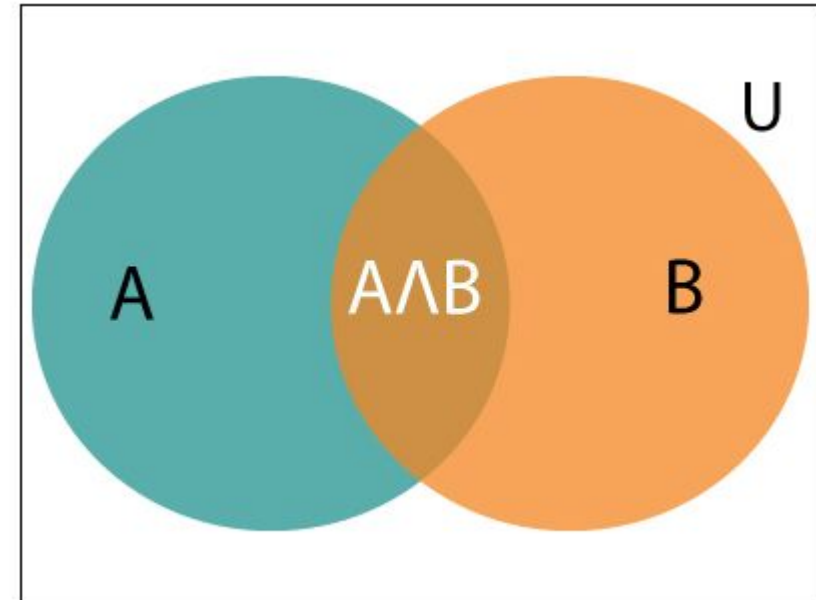
Where $P(A \cap B)$ = Joint probability of A and B

$P(B)$ = Marginal probability of B.

If the probability of A is given and we need to find the probability of B, then it will be given as:

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

It can be explained by using the below Venn diagram, where B is occurred event, so sample space will be reduced to set B, and now we can only calculate event A when event B is already occurred by dividing the probability of $P(A \cap B)$ by $P(B)$.



Example:

In a class, there are 70% of the students who like English and 40% of the students who likes English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let, A is an event that a student likes Mathematics

B is an event that a student likes English.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics.

Bayes' theorem:

- Bayes' theorem is also known as **Bayes' rule**, **Bayes' law**, or **Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.
 - Describes the probability of an event based on prior knowledge of conditions that might be related to the event.
 - In probability theory, it relates the conditional probability and marginal probabilities of two random events.
 - Bayes' theorem was named after the British mathematician **Thomas Bayes**.
 - The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.
 - It is a way to calculate the value of $P(B|A)$ with the knowledge of $P(A|B)$.
 - Bayes' theorem allows updating the probability prediction of an event by observing new information of the real world.
 - Bayes' theorem can be derived using product rule and conditional probability of event A with known event B
- As from product rule we can write:

$$P(A|B) = \frac{\text{Number of times A and B}}{\text{Number of times B}}$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

It is a way to calculate $P(A|B)$ with knowledge of $P(B|A)$

$$P(A \cap B) = P(A|B) \cdot P(B) \text{-----(I)}$$

$$P(A \cap B) = P(B|A) \cdot P(A) \text{-----(II)}$$

From 1 and 2 equation

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

It shows the simple relationship between joint and conditional probabilities. Here,

- $P(A|B)$ is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.
- $P(B|A)$ is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence.
- $P(A)$ is called the **prior probability**, probability of hypothesis before considering the evidence
- $P(B)$ is called **marginal probability**, pure probability of an evidence.

In general,

we can write $P(B) = \sum P(A_i) \cdot P(B|A_i)$, hence the Bayes' rule can be written as:

$$P(A_i|B) = \frac{P(A_i) \cdot P(B|A_i)}{\sum_{i=1}^k P(A_i) \cdot P(B|A_i)}$$

Where $A_1, A_2, A_3, \dots, A_n$ is a set of mutually exclusive and exhaustive events.

Joint probability distribution:

If we have variables $x_1, x_2, x_3, \dots, x_n$, then the probabilities of a different combination of $x_1, x_2, x_3 \dots x_n$, are known as Joint probability distribution.

$P[x_1, x_2, x_3, \dots, x_n]$, it can be written as the following way in terms of the joint probability distribution.

$$= P[x_1 | x_2, x_3, \dots, x_n] P[x_2, x_3, \dots, x_n]$$

$$= P[x_1 | x_2, x_3, \dots, x_n] P[x_2 | x_3, \dots, x_n] \dots P[x_{n-1} | x_n] P[x_n].$$

In general for each variable X_i , we can write the equation as:

$$P(X_i | X_{i-1}, \dots, X_1) = P(X_i | \text{Parents}(X_i))$$

Applying Bayes' rule:

Bayes' rule allows us to compute the single term $P(B|A)$ in terms of $P(A|B)$, $P(B)$, and $P(A)$. This is very useful in cases where we have a good probability of these three terms and want to determine the fourth one. Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

$$P(\text{cause} | \text{effect}) = \frac{P(\text{effect} | \text{cause}) P(\text{cause})}{P(\text{effect})}$$

Example-1:

Question: what is the probability that a patient has diseases dengue with a stiff neck?

Given Data:

A doctor is aware that disease dengue causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows:

The Known probability that a patient has dengue disease is 1/30,000.

The Known probability that a patient has a stiff neck is 2%.

Let a be the proposition that patient has stiff neck and b be the proposition that patient has dengue , so we can calculate the following as:

$$P(a|b) = 0.8$$

$$P(b) = 1/30000$$

$$P(a) = .02$$

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)} = \frac{0.8 * (\frac{1}{30000})}{0.02} = 0.0013333333.$$

Hence, we can assume that 1 patient out of 750 patients has meningitis disease with a stiff neck.

Example 2:Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is $4/52$, then calculate posterior probability $P(\text{King}|\text{Face})$, which means the drawn face card is a king card.

$$\mathbf{P(\text{king} | \text{face}) = \frac{P(\text{Face}|\text{king}) \cdot P(\text{King})}{P(\text{Face})} \dots\dots(i)}$$

$P(\text{king})$: probability that the card is King= $4/52 = 1/13$

$P(\text{face})$: probability that a card is a face card= $3/13$

$P(\text{Face}|\text{King})$: probability of face card when we assume it is a king = 1

Putting all values in equation (i)

we get

$$P(\text{king}|\text{face}) = 1 \cdot (1/13) / (3/13)$$

= $1/3$ It is a probability that a face card is a king card.

Application of Bayes' theorem in Artificial intelligence:

- It is used to calculate the next step of the robot when the already executed step is given.
- Bayes' theorem is helpful in weather forecasting.
- It can solve the Monty Hall problem.

Bayesian networks

- Bayesian belief network is key computer technology for dealing with probabilistic events and to solve a problem which has uncertainty.
- It defines probabilistic independencies and dependencies among the variables in the network.
- We can define a Bayesian network as: "A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."
- It is also called a **Bayes network**, **belief network**, **decision network**, or **Bayesian model**.
- Bayesian networks are probabilistic, because these networks are built from a **probability distribution**, and also use probability theory for prediction and anomaly detection.
- It consists of two parts
 - ✓ **Directed Acyclic Graph (DAG)**
 - ✓ **Table of conditional probabilities.**
- It can also be used in various tasks including **prediction**, **anomaly detection**, **diagnostics**, **automated insight**, **reasoning**, **time series prediction**, and **decision making under uncertainty**.
- Bayesian Network can be used for building models from data and experts opinions.
- The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an **Influence diagram**.

A Bayesian network graph is made up of nodes and Arcs (directed links), where:

- Each **node** corresponds to the random variables, and a variable can be **continuous** or **discrete**.
- **Arc or directed arrows** represent the causal relationship or conditional probabilities between random variables.
- These directed links or arrows connect the pair of nodes in the graph.

These links represent that one node directly influence the other node, and if there is no directed link that means that nodes are independent with each other

- **In the diagram, A, B, C, and D are random variables represented by the nodes of the network graph.**
- **If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.**
- **Node C is independent of node A.**

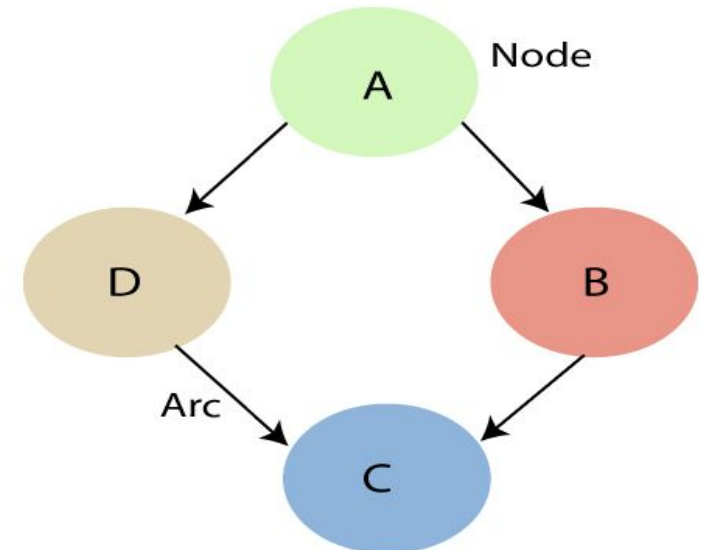


Table of conditional probabilities of all the nodes with effect of parent nodes.

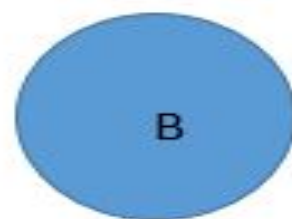
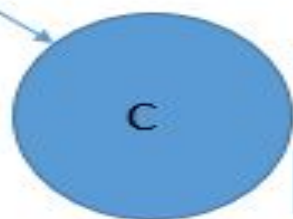
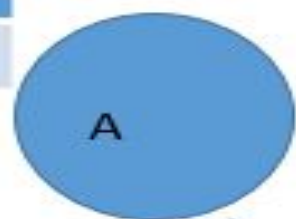
The Bayesian network has mainly two components:

- **Causal Component**
- **Actual numbers**

Each node in the Bayesian network has condition probability distribution $\mathbf{P(X_i | Parent(X_i))}$, which determines the effect of the parent on that node.

Example: To propagate in Bayesian network ,initial DAG is converted in to an undirected graph in which the arcs can be used to transmit probabilities in direction of evidence.

T	0.002
F	0.998



T	0.001
F	0.999

A	B	P(C=T)	P(C=F)
T	T		
F	T		
T	F		
F	F		

Example: Harry installed a new burglar alarm at his home to detect burglary. The alarm reliably responds at detecting a burglary but also responds for minor earthquakes. Harry has two neighbors David and Sophia, who have taken a responsibility to inform Harry at work when they hear the alarm. David always calls Harry when he hears the alarm, but sometimes he got confused with the phone ringing and calls at that time too. On the other hand, Sophia likes to listen to high music, so sometimes she misses to hear the alarm. Here we would like to compute the probability of Burglary Alarm.

Problem:

Calculate the probability that alarm has sounded, but there is neither a burglary, nor an earthquake occurred, and David and Sophia both called the Harry.

Solution:

- The Bayesian network for the above problem is given below. The network structure is showing that burglary and earthquake is the parent node of the alarm and directly affecting the probability of alarm's going off, but David and Sophia's calls depend on alarm probability.
- The network is representing that our assumptions do not directly perceive the burglary and also do not notice the minor earthquake, and they also not confer before calling.
- The conditional distributions for each node are given as conditional probabilities table or CPT.
- Each row in the CPT must be sum to 1 because all the entries in the table represent an exhaustive set of cases for the variable.
- In CPT, a Boolean variable with k Boolean parents contains 2^k probabilities. Hence, if there are two parents, then CPT will contain 4 probability values

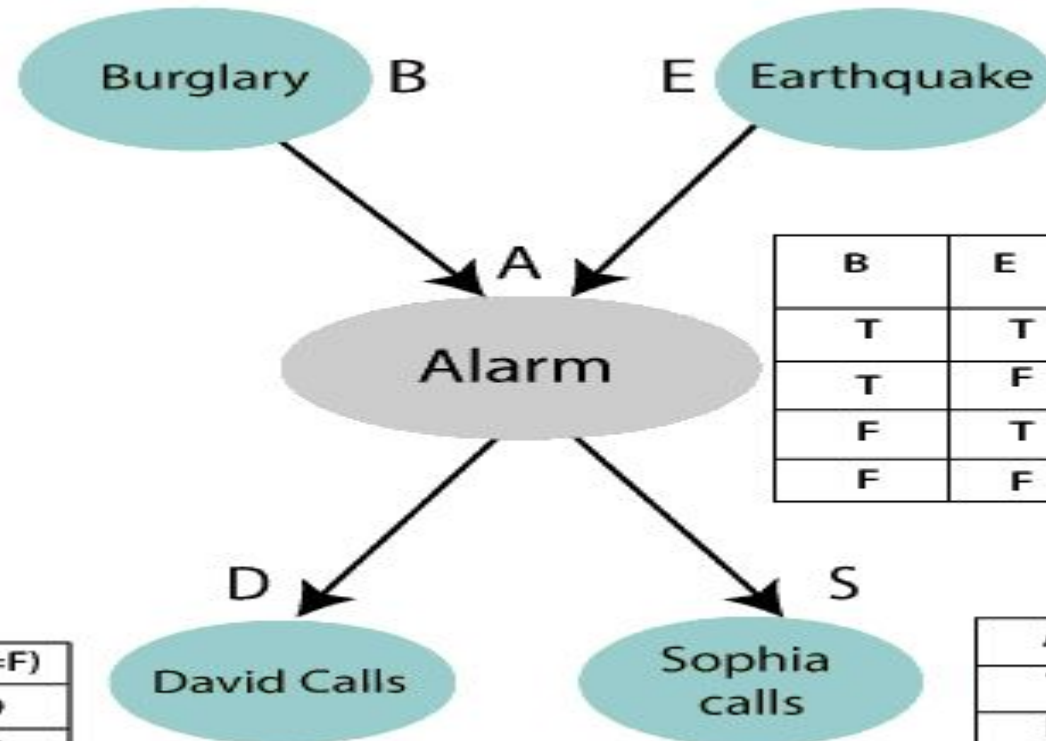
List of all events occurring in this network:

- Burglary (B)**
- Earthquake(E)**
- Alarm(A)**
- David Calls(D)**
- Sophia calls(S)**

We can write the events of problem statement in the form of probability: **P[D, S, A, B, E]**, can rewrite the above probability statement using joint probability distribution:

$$\begin{aligned}
P[D, S, A, B, E] &= P[D \mid S, A, B, E] \cdot P[S, A, B, E] \\
&= P[D \mid S, A, B, E] \cdot P[S \mid A, B, E] \cdot P[A, B, E] \\
&= P[D \mid A] \cdot P[S \mid A, B, E] \cdot P[A, B, E] \\
&= P[D \mid A] \cdot P[S \mid A] \cdot P[A \mid B, E] \cdot P[B, E] \\
&= P[D \mid A] \cdot P[S \mid A] \cdot P[A \mid B, E] \cdot P[B \mid E] \cdot P[E]
\end{aligned}$$

T	0.002
F	0.998



T	0.001
F	0.999

B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	0.69	0.69
F	F	0.999	0.999

A	P(D=T)	P(D=F)
T	0.91	0.09
F	0.05	0.95

A	P(S=T)	P(S=F)
T	0.75	0.25
F	0.02	0.98

Let's take the observed probability for the Burglary and earthquake component:
 $P(B = \text{True}) = 0.002$, which is the probability of burglary.
 $P(B = \text{False}) = 0.998$, which is the probability of no burglary.
 $P(E = \text{True}) = 0.001$, which is the probability of a minor earthquake
 $P(E = \text{False}) = 0.999$, Which is the probability that an earthquake not occurred.
We can provide the conditional probabilities as per the below tables:

Conditional probability table for Alarm A:

The Conditional probability of Alarm A depends on Burglar and earthquake:

B	E	P(A= True)	P(A= False)
True	True	0.94	0.06
True	False	0.95	0.04
False	True	0.31	0.69
False	False	0.001	0.999

Conditional probability table for David Calls:

The Conditional probability of David that he will call depends on the probability of Alarm.

A	P(D= True)	P(D= False)
True	0.91	0.09
False	0.05	0.95

Conditional probability table for Sophia Calls:

The Conditional probability of Sophia that she calls is depending on its Parent Node "Alarm."

A	P(S= True)	P(S= False)
True	0.75	0.25
False	0.02	0.98

From the formula of joint distribution, we can write the problem statement in the form of probability distribution:

$$P(S, D, A, \neg B, \neg E) = P(S|A) * P(D|A) * P(A|\neg B \wedge \neg E) * P(\neg B) * P(\neg E).$$

$$= 0.75 * 0.91 * 0.001 * 0.998 * 0.999$$

$$= 0.00068045.$$

Hence, a Bayesian network can answer any query about the domain by using Joint distribution.

Certainty factors & Rule-based systems

Certainty factors and rule-based systems are concepts used in the field of artificial intelligence (AI) to represent and reason about uncertain knowledge and make decisions based on rules.

1. Certainty Factors:

- Certainty factors, also known as confidence factors or belief factors, are numerical values that represent the degree of certainty or confidence in the truth or falsity of a statement.
- They are used to handle uncertainty and partial knowledge in AI systems.
- In a certainty factor-based system, each rule has an associated certainty factor, typically ranging from -1.0 to +1.0.
- A positive certainty factor represents a degree of belief in the rule being true, while a negative certainty factor represents a degree of belief in the rule being false.
- A certainty factor of 0.0 indicates complete uncertainty.
- When multiple rules are applied to a given situation, their certainty factors are combined to derive an overall certainty factor for the conclusion or decision.
- Various methods, such as the weighted sum or Dempster-Shafer theory, can be used for combining certainty factors.
- Certainty factors provide a way to reason with uncertain or incomplete information and make decisions based on the available evidence and expert knowledge.
- They have been used in various AI applications, including expert systems and decision support systems.

Here are a few examples of how certainty factors can be applied:

1. Medical Diagnosis:

In a medical diagnosis system, certainty factors can be used to represent the degree of confidence in a particular disease or condition based on observed symptoms and test results. For example:

- If the symptom "high fever" is present, with a certainty factor of +0.8, it increases the likelihood of an infectious disease.
- If the symptom "rash" is present, with a certainty factor of +0.7, it suggests the possibility of a viral infection.

By combining certainty factors of symptoms and matching them against diagnostic rules, a system can generate a diagnosis with an overall certainty factor.

2. Risk Assessment:

Certainty factors can be used in risk assessment systems to evaluate the probability and severity of potential risks. For example, in a cyber security risk assessment:

- If a vulnerability is discovered, with a certainty factor of -0.7, it indicates a significant risk of a security breach.
- If appropriate security controls are in place, with a certainty factor of +0.6, it reduces the likelihood and impact of a successful attack.

By considering certainty factors associated with various risk factors, the system can determine an overall risk score.

3. Decision Support Systems:

Certainty factors can be employed in decision support systems to evaluate different options and guide decision-making. For example, in an investment decision support system:

- If the economic outlook is positive, with a certainty factor of +0.8, it increases the confidence in the potential returns of an investment.
- If a particular industry is experiencing significant regulatory challenges, with a certainty factor of -0.6, it decreases the attractiveness of investing in that industry.

By aggregating certainty factors associated with different factors influencing the decision, the system can provide recommendations or rankings for different investment options.

4. Fault Diagnosis:

Certainty factors can be used in fault diagnosis systems to identify the root causes of problems in complex systems. For example, in a machinery fault diagnosis system:

- If abnormal vibrations are detected, with a certainty factor of +0.7, it indicates a potential issue with the bearings.
- If abnormal noise is observed, with a certainty factor of +0.6, it suggests a possible problem with the motor.

By considering the certainty factors associated with different symptoms and matching them against diagnostic rules, the system can determine the most probable root cause of the fault.

Certainty factors can be used to represent and reason about uncertainty in different applications. By assigning and combining certainty factors, AI systems can make more informed decisions and provide better explanations for their conclusions.

2. Rule-Based Systems:

- A rule-based system, also known as a production system, is an AI system that operates based on a set of predefined rules or logical statements.
- These rules are typically represented as "if-then" statements, where the "if" part specifies conditions or patterns, and the "then" part defines actions or conclusions.
- The rule-based system uses a rule engine or inference engine to match the incoming data or facts against the rules and execute the corresponding actions.
- The system follows a top-down approach, where it applies rules in a sequential manner until a termination condition is met or a desired outcome is achieved.
- Rule-based systems provide a declarative and modular approach to represent and reason about knowledge.
- They are widely used in expert systems, decision support systems, and various AI applications, including diagnostic systems, intelligent tutoring systems, and expert advisors.
- The rules in a rule-based system can be augmented with certainty factors to handle uncertainty and prioritize conflicting rules. Certainty factors can help in selecting the most appropriate rule or resolving conflicts between rules when multiple rules are applicable.

- Overall, certainty factors and rule-based systems are valuable techniques in AI for handling uncertainty, representing knowledge, and making decisions based on logical rules.
- They have been used successfully in various domains to solve complex problems and assist human experts in decision-making tasks.

Here are a few examples of rule-based systems:

1. Expert Systems:

Expert systems are AI systems designed to mimic human expertise in a specific domain. They employ rule-based systems to capture the knowledge and decision-making processes of human experts. For example, a medical diagnosis expert system can use rules to analyze patient symptoms, medical history, and test results to provide a diagnosis and recommend appropriate treatments.

2. Fraud Detection Systems:

Rule-based systems are commonly used in fraud detection applications. Rules can be defined to identify patterns and behaviors that are indicative of fraudulent activities. For instance, a credit card fraud detection system may have rules that trigger alerts when certain criteria, such as unusual transaction amounts, locations, or purchasing patterns, are met.

3. Intelligent Tutoring Systems:

Intelligent tutoring systems assist learners by providing personalized instruction and feedback. Rule-based systems are used to model the learning domain and determine appropriate instructional strategies. Rules can be designed to adapt the content, difficulty level, and feedback based on the learner's performance and individual needs.

4. Workflow Automation:

Rule-based systems are employed in workflow automation to automate business processes and decision-making. Rules define the conditions and actions for each step in the workflow. For example, in an invoice processing system, rules can be set to determine the approval hierarchy based on the invoice amount and route the invoice to the appropriate individuals for review and authorization.

5. Diagnosis and Troubleshooting Systems:

Rule-based systems are used in diagnostic and troubleshooting applications to identify problems and recommend solutions. Rules capture the known symptoms, causes, and resolutions in a domain. For instance, in a network troubleshooting system, rules can help identify network faults based on observed symptoms and suggest steps to resolve the issues.

Rule-based systems provide a flexible and modular approach to automate decision-making and problem-solving tasks based on predefined rules and logical reasoning.

Weak slot and filler structures

In the context of natural language processing and information extraction, "slot and filler" structures, also known as "slot-value pairs," are used to represent structured information extracted from text.

A "slot" refers to a specific attribute or property of an entity, while a "filler" represents the value or content associated with that attribute. The concept of weak slot and filler structures relates to the presence of uncertainty or incomplete information in these representations.

In weak slot and filler structures, some slots may have missing or uncertain fillers. This means that the exact value or content of a particular attribute is unknown or not fully specified. Instead of providing a specific value, a weak slot and filler structure may include alternative possibilities, probabilistic estimates, or other indicators of uncertainty.

Here's an example to illustrate the concept:

Text: "The city's annual rainfall in 2022 was around [weak slot: amount] 800-900 millimeters."

In this example, the slot is "amount," representing the attribute of rainfall, and the filler is "800-900 millimeters," indicating a range of possible values for the annual rainfall in 2022. The weak slot and filler structure suggests that the exact amount is uncertain, and it could fall within that range.

Weak slot and filler structures are particularly useful when dealing with ambiguous or incomplete information in text. They allow for representing uncertainty, variability, or lack of precise knowledge about certain attributes. This approach acknowledges that information extraction from natural language can be challenging, and not all attributes can be determined with absolute certainty.

By incorporating weak slot and filler structures, information extraction systems can provide a more nuanced and flexible representation of extracted information, accommodating uncertain or incomplete data points and allowing for further reasoning and decision-making processes.

- Semantic nets
- Frames

Semantic Nets:

The idea behind a semantic network is that knowledge is often best understood as a set of concepts that are related to one another.

A semantic network consists of nodes that are connected by labeled arcs. The nodes represent concepts and the arcs represent relation between concepts. A fragment of a typical semantic net is shown below .

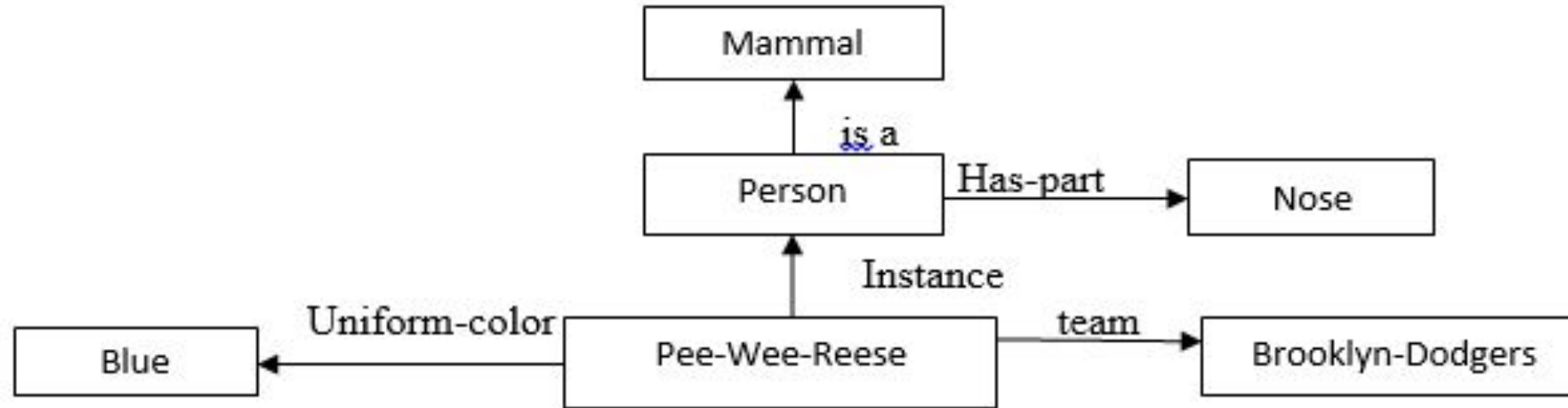


Fig. 9.1. A Semantic Network

This network contains examples of both the is a and instance relations, as well as some other, more domain specific relations like team and Uniform color.

Common semantic relations

INSTANCE: X is an INSTANCE of Y if X is a specific example of the general concept Y.

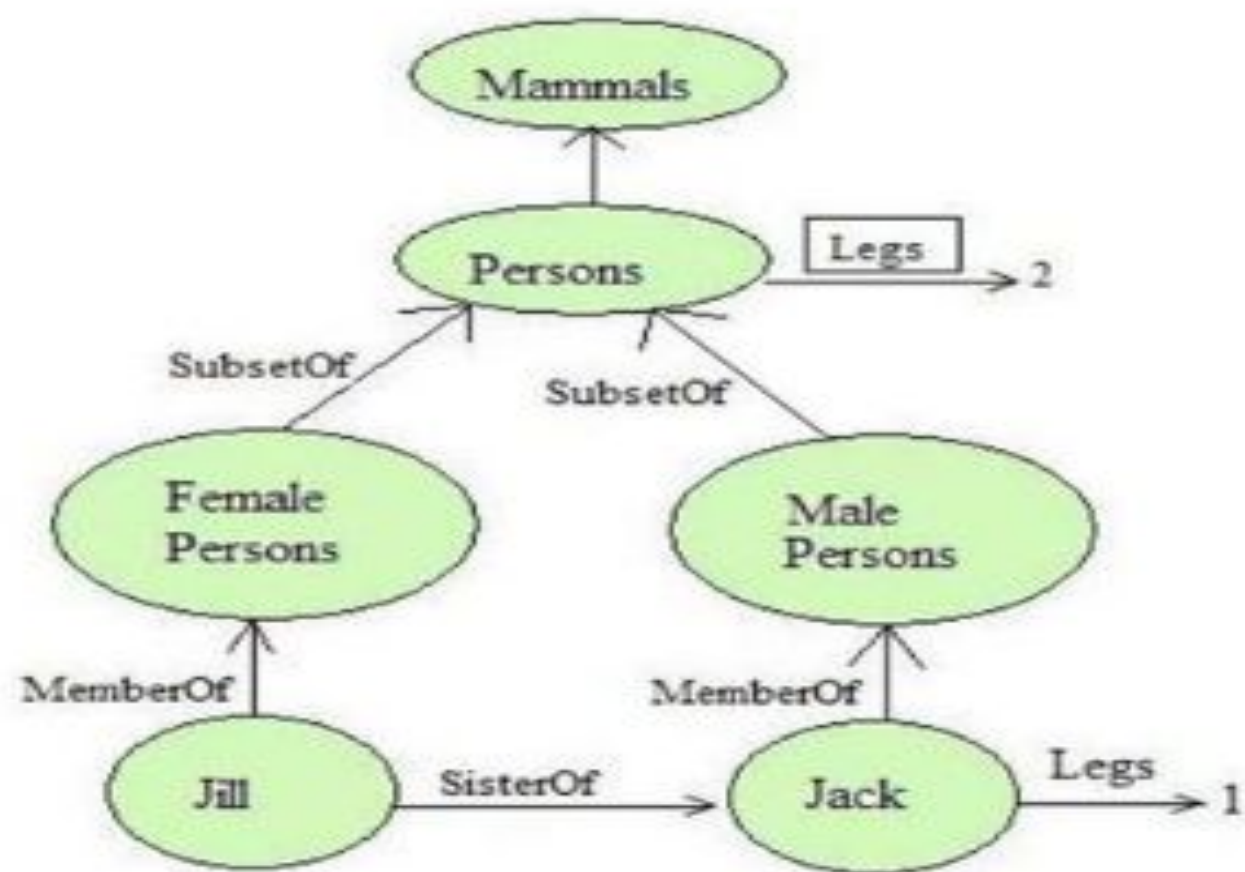
Example: *Elvis* is an **INSTANCE** of *Human*

ISA: X ISA Y if X is a subset of the more general concept Y.

Example: *sparrow* **ISA** *bird*

HASPART: X HASPART Y if the concept Y is a part of the concept X. (Or this can be any other property)

Example: *sparrow* **HASPART** *tail*



□ Intersection Search

One of the early ways that semantic nets were used was to find relationships among objects by spreading activation out from each of two nodes and seeing where the activation met. This process is called intersection

Using this process, it is possible to use the network of Fig.9.1. to answer questions such as

“What is the connection between the Brooklyn Dodgers and blue?”

This kind of reasoning exploits one of the important advantages that slot-and-filler structures have over purely logical representations because it takes advantage of entity-based organization of knowledge that slot-and-filler representation provide.

This network contains examples of both the is-a and instance relations, as well as some other, more domain-specific relations like team and uniform-color. In this network, we could use inheritance to derive the additional relation.

Has-part(Pee-Wee-Reese, Nose)

□ Representing Non binary Predicates

Semantic nets are a natural way to represent relationships that would appear as ground instances of binary predicates in predicate logic.

- Unary predicates can be written as binary ones.

Man (Marcus)

could be rewritten as

Instance (Marcus, Man)

- N-Place Predicates

Score(Cubs,Dodgers,5-3)

Becomes

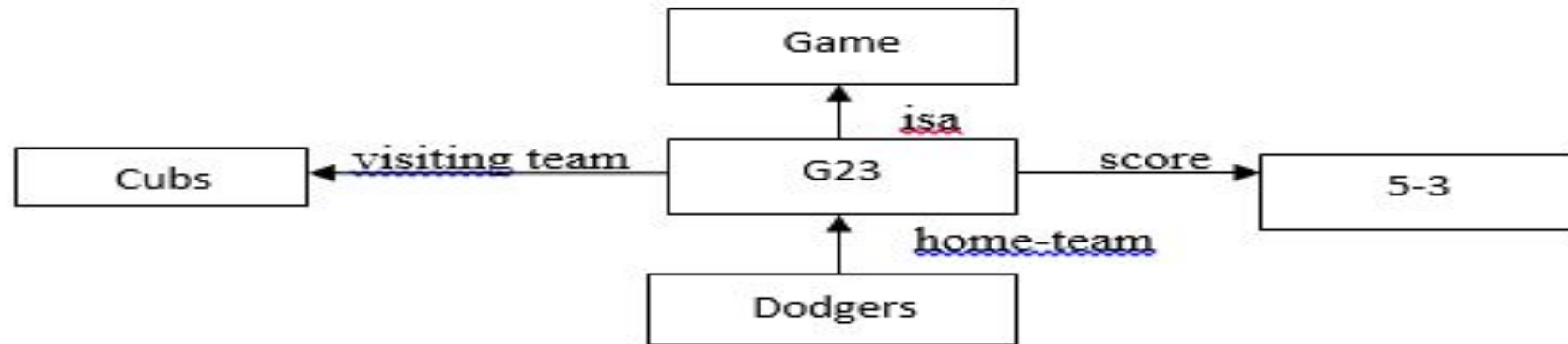


Fig. 9.2. A Semantic Net for an n-Place Predicate

Semantic net representation as a sentence

This technique is particularly useful for representing the contents of a typical declarative sentence that describes several aspects of a particular event. The sentence

John gave the book to Mary

Could be represented by the network shown in Fig. 9.3. In fact, several of the earliest uses of semantic nets were in English-understanding programs.

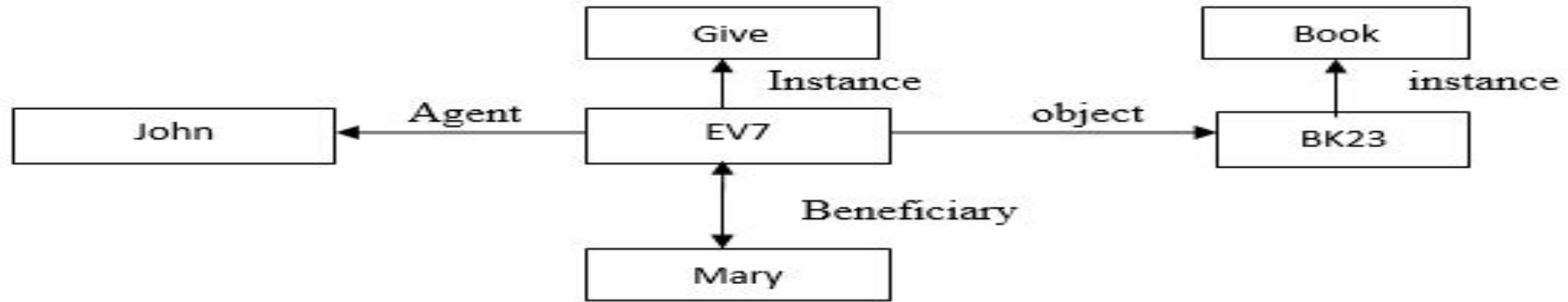


Fig. 9.3. A Semantic Net Representing a Sentence

□ Some Important Distinctions:

● First try :



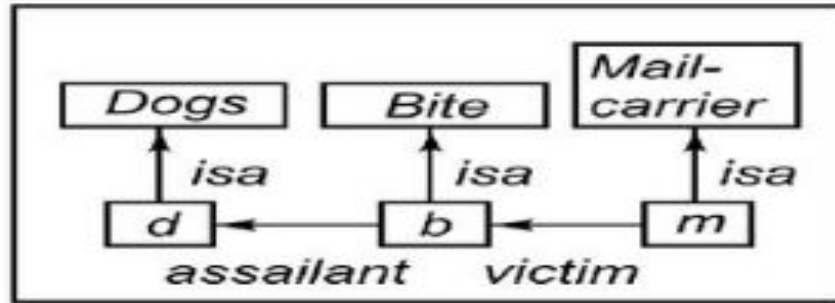
● Second try :



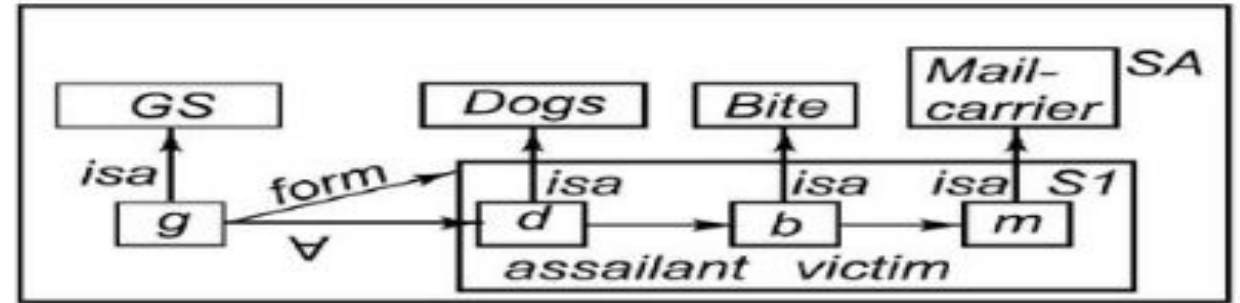
● Third try :



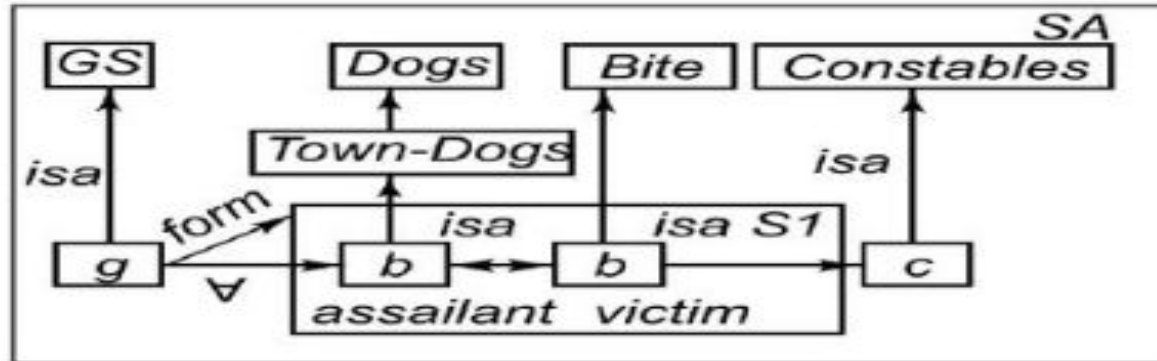
□ Partitioned semantic nets



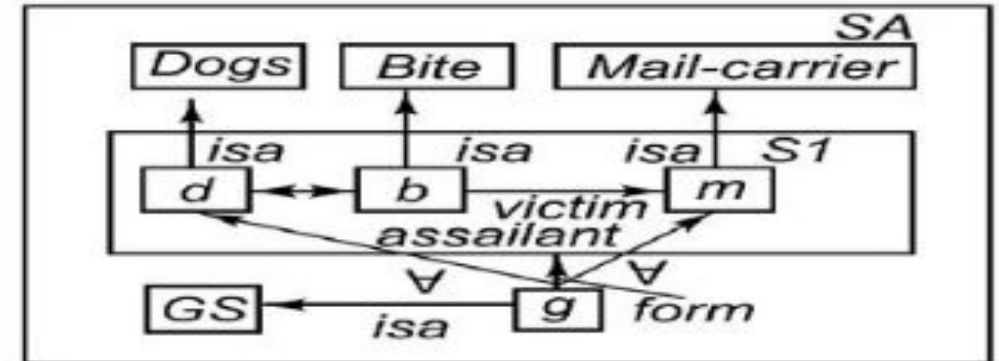
(a)



(b)



(c)



(d)

- a) The dog bit the mail carrier.
- b) Every dog has bitten a mail carrier.
- c) Every dog in town has bitten the constable.
- d) Every dog has bitten every mail carrier.

Advantages:

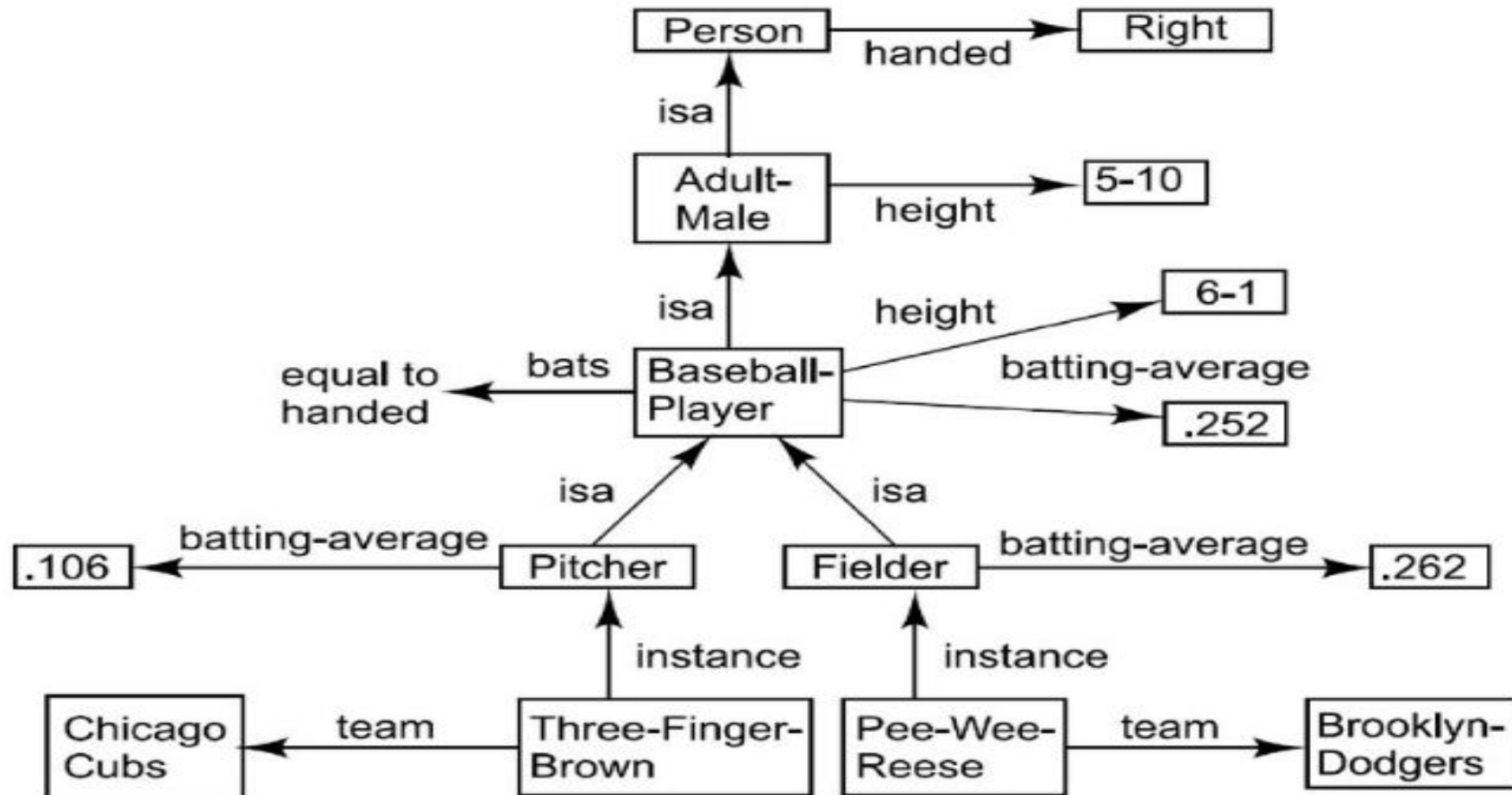
- Semantic nets have the ability to represent default values for categories.
- Convey some meaning in a transparent manner.
- Simple and easy to understand.
- Easy to translate into PROLOG.

Disadvantages:

- Links between the objects represents only binary relations. The sentence Run(Chennai Express,Chennai,Bangalore,Today) cannot be asserted directly.
- Some types of properties are not easily . For example: negation, disjunction and general non taxonomic knowledge. Expresses using a semantic network.
- There is no standard definition of links.

Frames

- A frame represents an entity as a set of slots (attributes) and associated values.
- Each slot may have constraints that describe legal values that the slot can take.
- A frame can represent a specific entity, or a general concept.
- Frames are implicitly associated with one another because the values of a slot can be another frame.



A Simplified Frame System

Person

isa : *Mammal*
cardinality : 6,000,000,000
** handed :* *Right*

Adult-Male

isa : *Person*
cardinality : 2,000,000,000
** height :* 5-10

ML-Baseball-Player

isa : *Adult-Male*
cardinality : 624
** height :* 6-1
** bats :* equal to handed
** batting-average :* .252
** team :*
** uniform-color :*

Fielder

isa : *ML-Baseball-Player*

cardinality : 376

** batting-average :* .262

Pee-Wee-Reese

instance : *Fielder*

height : 5-10

bats : *Right*

batting-average : .309

team : *Brooklyn-Dodgers*

uniform-color : *Blue*

ML-Baseball-Team

isa: *Team*

cardinality : 26

** team-size :* 24

** manager :*

Brooklyn-Dodgers

instance :

team-size :

manager :

players :

ML-Baseball-Team

24

Leo-Durocher

{Pee-Wee-Reese,...}

Representing the class of all teams as a metaclass

Class

instance : *Class*
isa : *Class*
** cardinality :*

Team

instance : *Class*
isa : *Class*
cardinality : {the number of teams that exist}
**team-size :* {each team has a size}

ML-Baseball-Team

isa : *Mammal*
instance : *Class*
isa : *Team*
cardinality : 26 {the number of baseball teams that exist}
** team-size :* 24 {default 24 players on a team}
** manager :*

Representing the class of all teams as a metaclass (Cont..)

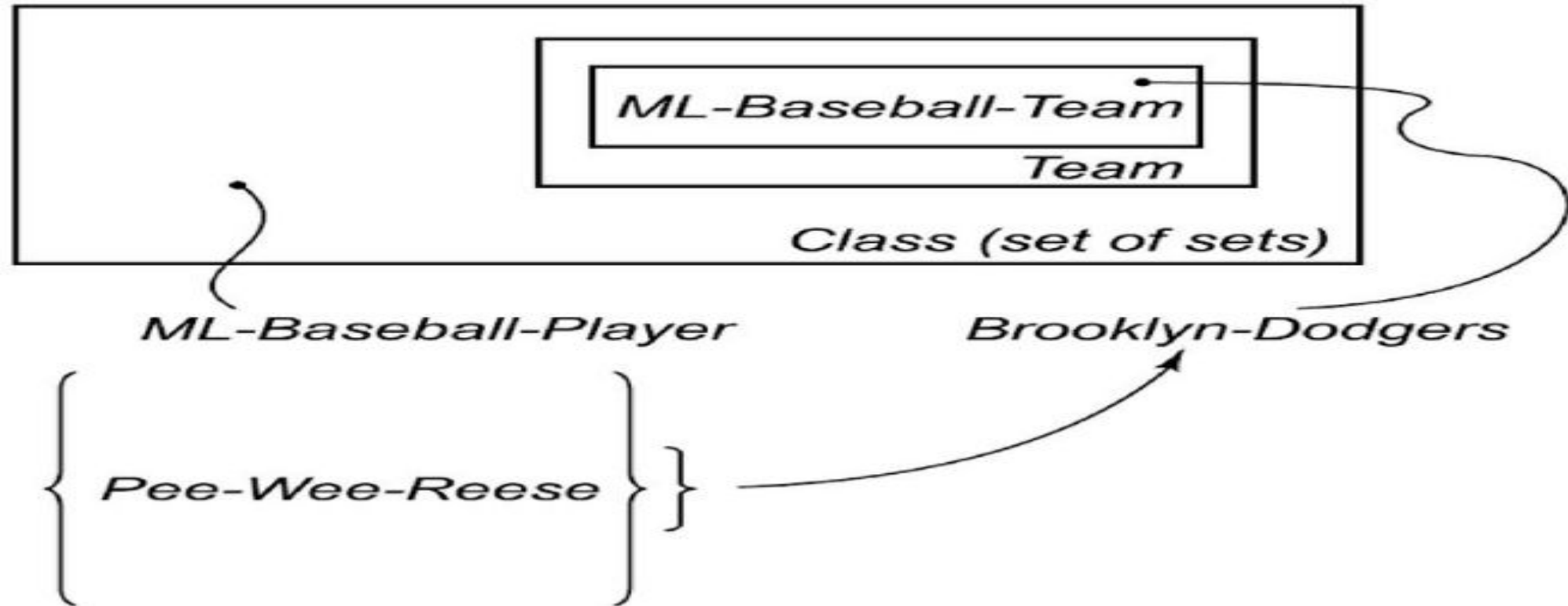
Brooklyn-Dodgers

<i>instance :</i>	<i>ML-Baseball-Team</i>
<i>isa :</i>	<i>ML-Baseball-Player</i>
<i>team-size :</i>	<i>24</i>
<i>manager :</i>	<i>Leo-Durocher</i>
<i>* uniform-color :</i>	<i>Blue</i>

Pee-Wee-Reese

<i>instance :</i>	<i>Brooklyn-Dodgers</i>
<i>instance :</i>	<i>Fielder</i>
<i>uniform-color :</i>	<i>Blue</i>
<i>batting-average :</i>	<i>.309</i>

Classes and Metaclasses

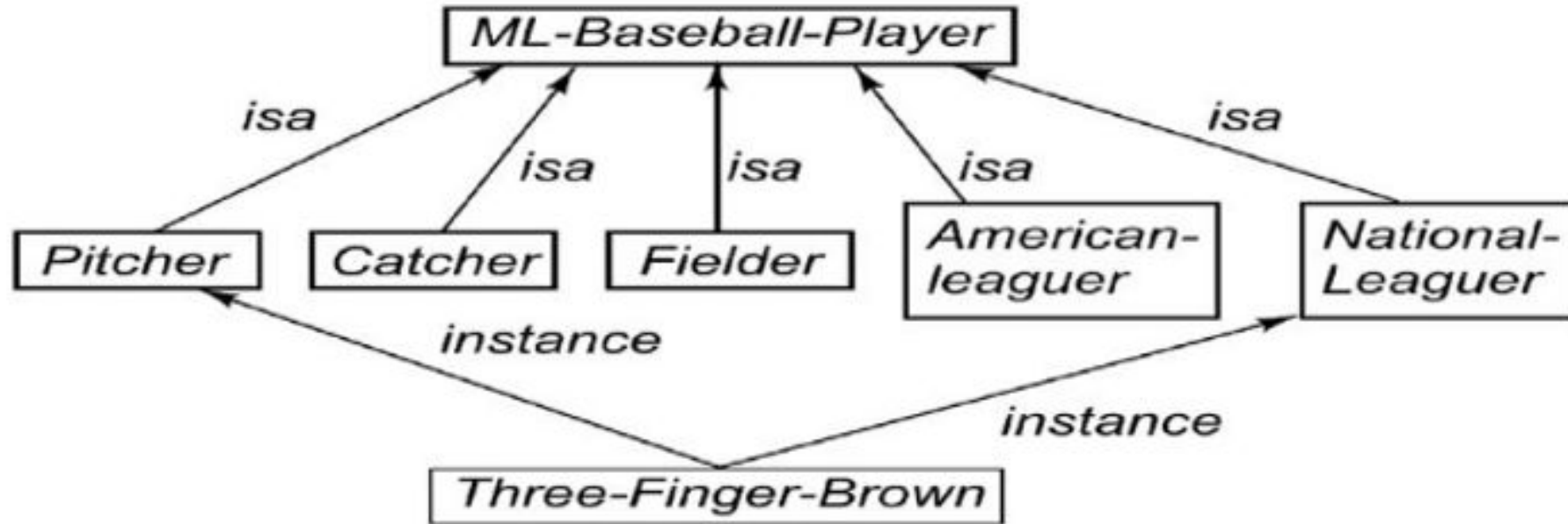


Slots as full –fledged objects

We want to be able to represent and use the following properties of slots (attributes or relations)

- The classes to which the attribute can be attached.
- Constraints to either the type or the value of the attribute.
- A value that all instances of a class must have by the definition of the class.
- A default value for the attribute.
- Rules for inheriting values from the attributes.
- Rules for computing a value separately from inheritance.
- An inverse attribute.
- Whether the slot is single valued or multivalued.

Representing relationships among classes



ML-Baseball-Player
is-covered-by :

{Pitcher, Catcher, Fielder}
{American-Leaguer, National-Leaguer}

Pitcher

isa :

mutually-disjoint-with :

ML-Baseball-Player
{Catcher, Fielder}

Continued...

Catcher

isa :

ML-Baseball-Player

mutually-disjoint-with:

{Pitcher, Fielder}

Fielder

isa :

ML-Baseball-Player

mutually-disjoint-with :

{Pitcher, Catcher}

American-Leaguer

isa :

ML-Baseball-Player

mutually-disjoint-with :

{National-Leaguer}

National-Leaguer

isa :

ML-Baseball-Player

mutually-disjoint-with :

{American-Leaguer}

Three-Finger-Brown

instance :

Pitcher

instance :

National-Leaguer

Strong slot and filler structures

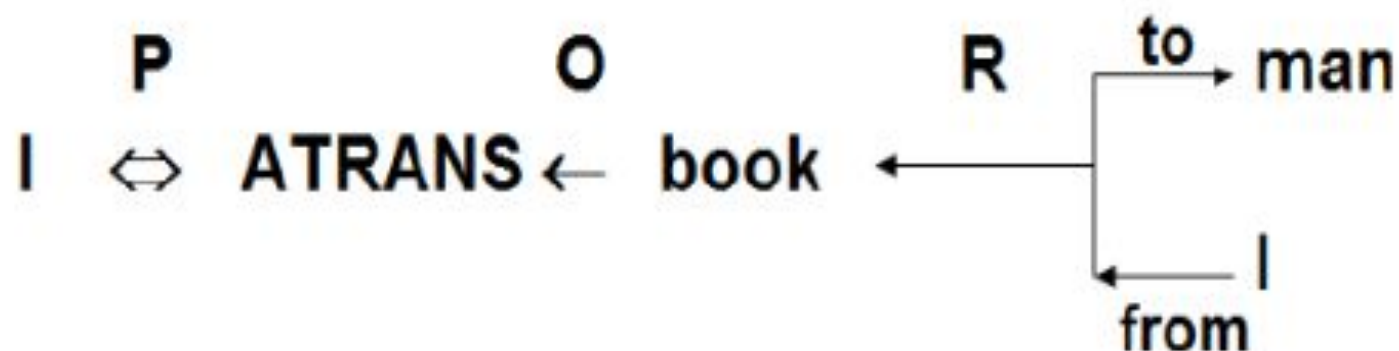
Conceptual Dependency

- Conceptual dependency(CD) is a theory of how to represent a kind of knowledge about events that is usually contained in natural language sentence.
- The goal is to represent knowledge in a way that;
 - ✓ To help drawing inferences from the sentence.
 - ✓ To independent of the language in which the sentences were originally stated.
 - ✓ Capable of read and understand natural language text.
- It has been used by many programs that portend to understand English.
- CD Provides
 - Provides both a structure and a specific set of primitives
 - A particular level of granularity, out of which representations of particular pieces of information can be constructed.

- Sentences are represented as a series of diagrams depicting actions using both abstract and real physical situations.
 - The agent and the objects are represented.
 - The actions are built up from a set of primitive acts which can be modified by tense.
- CD is based upon events and actions. Every event (if applicable) has:
 - an ACTOR
 - an ACTION performed by the Actor
 - an OBJECT that the action performs on
 - a DIRECTION in which that action is oriented
- These are represented as slots and fillers. In English sentences, many of these attributes are left out.

A Simple Conceptual Dependency Representation

- For the sentences, "I gave a book to the man" CD representation is as follows:



- Where the symbols have the following meaning.
 - Arrows indicate directions of dependency.
 - Double arrow indicates two way link between actor and action.
 - O -- for the object case relation
 - R – for the recipient case relation
 - P – for past tense
 - D - destination

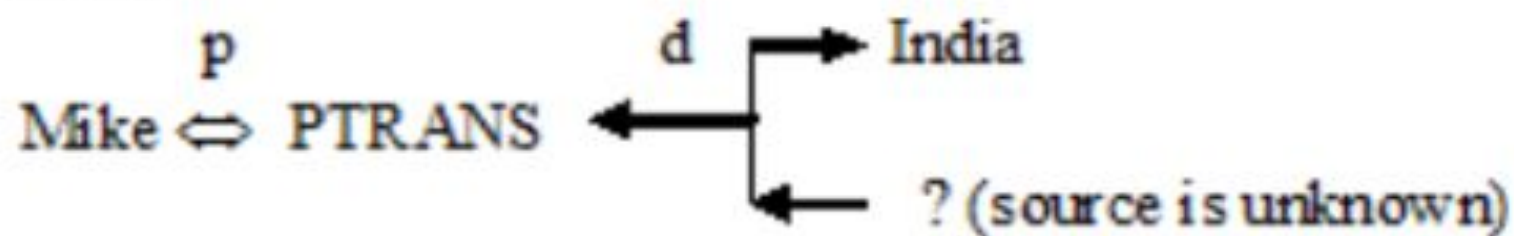
Primitive Acts of CD theory

ATRANS	Transfer of an abstract relationship (i.e. give)
PTRANS	Transfer of the physical location of an object (e.g., go)
PROPEL	Application of physical force to an object (e.g. push)
MOVE	Movement of a body part by its owner (e.g. kick)
GRASP	Grasping of an object by an action (e.g. throw)
INGEST	Ingesting of an object by an animal (e.g. eat)
EXPEL	Expulsion of something from the body of an animal (e.g. cry)
MTRANS	Transfer of mental information (e.g. tell)
MBUILD	Building new information out of old (e.g. decide)
SPEAK	Producing of sounds (e.g. say)
ATTEND	Focusing of a sense organ toward a stimulus (e.g. listen)

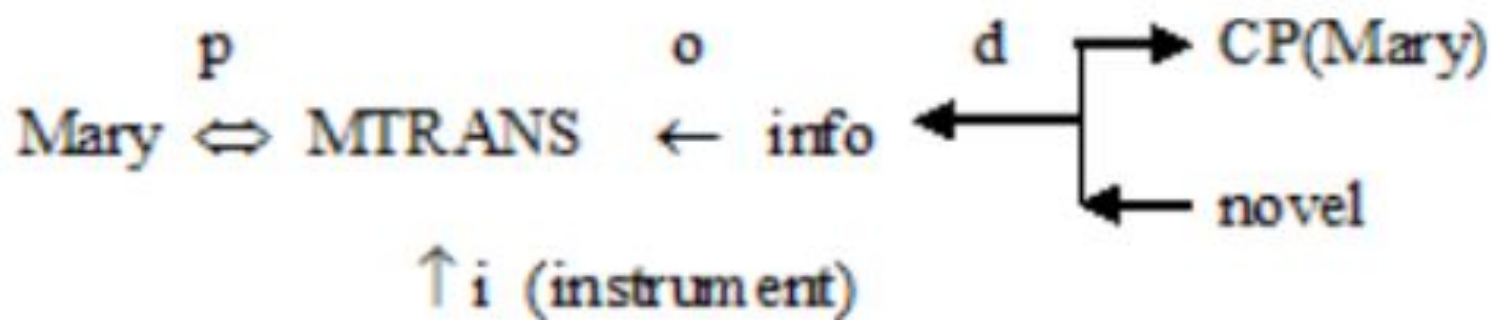
- There are four conceptual categories. These are,

ACT	Actions {one of the CD primitives}
PP	Objects {picture producers}
AA	Modifiers of actions {action aiders}
PA	Modifiers of PP's {picture aiders}

Example 1: Mike went to India



Example 2: Mary read a novel



- The primitives are used to define conceptual dependency relationships or Conceptual syntax rules.

$PP \Leftrightarrow ACT$ indicates that an actor acts.

$PP \Leftrightarrow PA$ indicates that an object has a certain attribute.

$\overset{O}{ACT} \leftarrow PP$ indicates the object of an action.

$ACT \leftarrow \overset{R}{\begin{array}{l} \rightarrow PP \\ \leftarrow PP \end{array}}$ indicates the recipient and the donor of an object within an action.

$ACT \leftarrow \overset{D}{\begin{array}{l} \rightarrow PP \\ \leftarrow PP \end{array}}$ indicates the direction of an object within an action.

ACT $\xleftarrow{1}$ $\updownarrow$

indicates the instrumental conceptualization for an action.

X
 $\updownarrow$
Y

indicates that conceptualization X caused conceptualization Y.
When written with a C this form denotes that X COULD cause Y.

PP $\leftarrow$ $\begin{cases} \rightarrow \text{PA2} \\ \leftarrow \text{PA1} \end{cases}$

indicates a state change of an object.

PP1 $\leftarrow$ PP2

indicates that PP2 is either PART OF or the POSSESSOR OF PP1.

CD Conceptual Tenses

p	Past
f	Future
t	Transition
t_s	Start transition
t_f	Finished transition
k	Continuing
?	Interrogative
/	Negative
nil	Present
delta	Timeless
c	Conditional

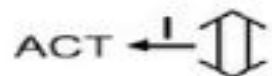
The Dependencies of CD



- | | | | |
|----|--|---|----------------------------------|
| 1. | PP $\longleftrightarrow$ ACT | John $\overset{p}{\longleftrightarrow}$ PTRANS | John ran. |
| 2. | PP $\Longleftrightarrow$ PA | John $\Longleftrightarrow$ height (> average) | John is tall. |
| 3. | PP $\Longleftrightarrow$ PA | John $\Longleftrightarrow$ doctor | John is a doctor. |
| 4. | PP
↑
PA | boy
↑
nice | A nice boy. |
| 5. | PP
↑↑
PP | dog
↑↑ Poss-by
John | John's dog. |
| 6. | ACT $\overset{o}{\longleftarrow}$ PP | John $\overset{p}{\longleftrightarrow}$ PROPEL $\overset{o}{\longleftarrow}$ cart | John pushed the cart. |
| 7. | ACT $\overset{o}{\longleftarrow}$ $\left\{ \begin{array}{l} \rightarrow \text{PP} \\ \leftarrow \text{PP} \end{array} \right.$ | John $\overset{p}{\longleftrightarrow}$ ATRANS $\overset{o}{\longleftarrow}$ $\left\{ \begin{array}{l} \rightarrow \text{John} \\ \leftarrow \text{Mary} \end{array} \right.$
↑
book | John took the book from Mary. |
| 8. | ACT $\overset{l}{\longleftarrow}$ $\left\{ \begin{array}{l} \uparrow \\ \uparrow \end{array} \right.$ | John $\overset{p}{\longleftrightarrow}$ INGEST $\overset{l}{\longleftarrow}$ $\left\{ \begin{array}{l} \uparrow \text{John} \\ \uparrow \text{do} \\ \uparrow \text{spoon} \end{array} \right.$
↑
ice cream | John ate ice cream with a spoon. |

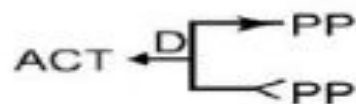
The Dependencies of CD (Cont'd)

8.



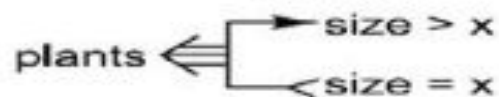
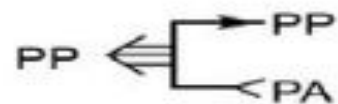
John ate ice cream with a spoon.

9.



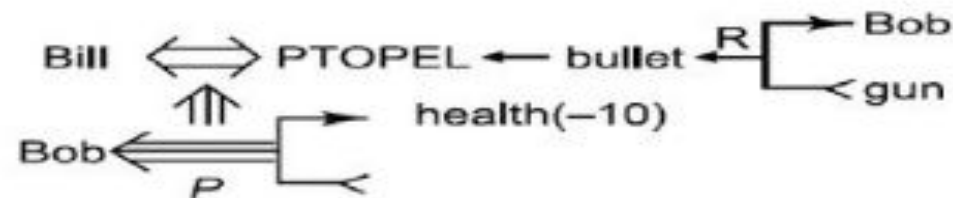
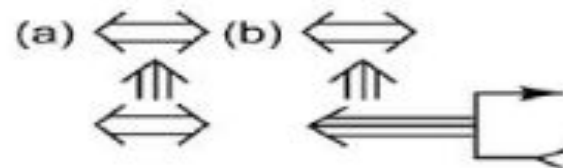
John fertilized the field.

10.



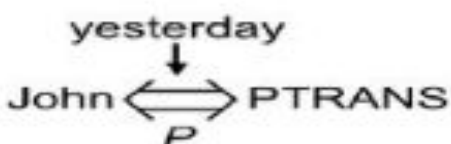
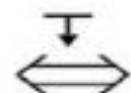
The plants grew.

11.



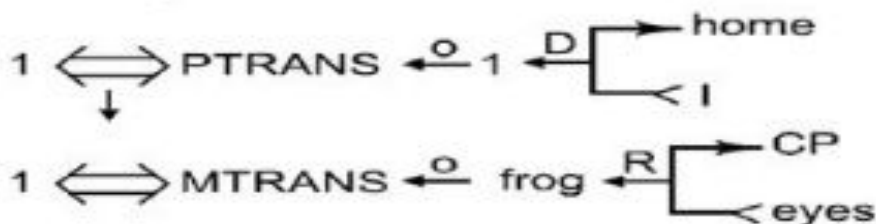
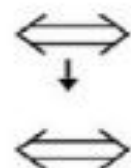
Bill shot Bob.

12.



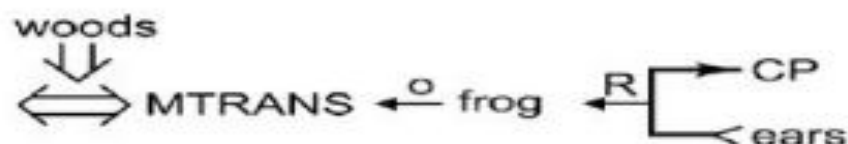
John ran yesterday.

13.



While going home, I saw a frog.

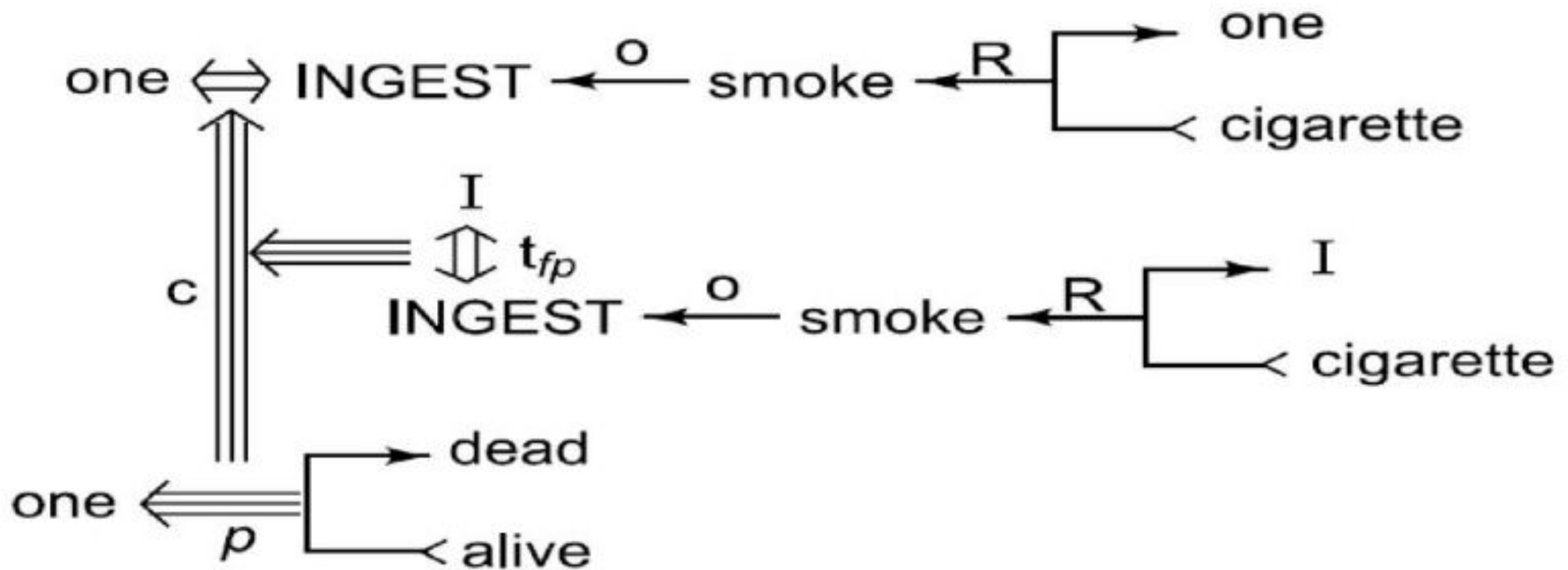
14.



I heard a frog in the woods.

Using Conceptual Tenses

- “Since smoking can kill you, I stopped.”



Advantages of CD

- Using these primitives involves fewer inference rules.
- Many inference rules are already represented in CD structure.
- The holes in the initial structure help to focus on the points still to be established.

Dis advantages of CD

- Knowledge must be composed into fairly low level primitives.
- Impossible or difficult to find correct set of primitives.
- A lot of inference may still be required.
- Representations can be complex even for relatively simple actions.
- Complex representation required a lot of storage.

Scripts

- A script is a structure that describes a stereotyped sequence of events in a particular context.
 - Associated with each slot may be some information about what kinds of values it may contain as well as default value to be used if no other information is available.
-
- A script is a structure that prescribes a set of circumstances which could be expected to follow on from one another.
 - It is similar to a thought sequence or a chain of situations which could be anticipated.
 - It could be considered to consist of a number of slots or frames but with more specialized roles.
 - Scripts are beneficial because:
 - Events tend to occur in known runs or patterns.
 - Causal relationships between events exist.
 - Entry conditions exist which allow an event to take place
 - Prerequisites exist upon events taking place. E.g. when a student progresses through a degree scheme or when a purchaser buys a house.

Various components of Script are

- **Script Name:** - Title
- **Track:** - Special situation, specific variation
- **Roles:-** peoples involve in the event described in script
- **Entry condition:** - required pre situation to execute the script
- **Props:** - non live object involve in the Script
- **Scenes:** - The actual sequence of events that occur
- **Result:** - Condition that will be True after events described in the script are occurred

Scripts are useful

- The events described in a script form a **giant causal chain**.



- If a particular script is known to be appropriate in a given situation then it can be very useful in predicting the occurrence of events that were not explicitly mentioned.
- Scripts indicates how events which are mentioned relate to each other.
- Before a particular script can be applied it must be activated.
Fleeting scripts
Non fleeting scripts

Script: RESTAURANT Track: Coffee Shop Props: Tables Menu F = Food Check Money Roles: S = Customer W = Waiter C = Cook M = Cashier O = Owner	Scene 1: Entering S PTRANS S into restaurant S ATTEND eyes to tables S MBUILD where to sit S PTRANS S to table S MOVE S to sitting position Scene 2: Ordering (Menu on table) (W brings menu) S PTRANS menu to S (S asks for menu) S MTRANS signal to W W PTRANS W to table S MTRANS 'need menu' to W W PTRANS W to menu W PTRANS W to table W ATRANS menu to S S MTRANS W to table * S MBUILD choice of F S MTRANS signal to W W PTRANS W to table S MTRANS 'I want F' to W W PTRANS W to C W MTRANS (ATRANS F) to C C MTRANS 'no F' to W W PTRANS W to S W MTRANS 'no F' to S (go back to *) or (go to Scene 4 at no pay path) C DO (prepare F script) to Scene 3
Entry conditions: S is hungry. S has money. Results: S has less money. O has more money. S is not hungry. S is pleased (optional).	Scene 3: Eating C ATRANS F to W W ATRANS F to S S INGEST F (Option: Return to Scene 2 to order more; otherwise, go to Scene 4) Scene 4: Exiting S MTRANS to W (W ATRANS check to S) W MOVE (write check) W PTRANS W to S W ATRANS check to S S ATRANS tip to W S PTRANS S to M S ATRANS money to M (No pay path) S PTRANS S to out of restaurant

Script Example

Script : Play in theater	Various Scenes
Track: Play in Theater Props: • Tickets • Seat • Play Roles: • Person (who wants to see a play) – P • Ticket distributor – TD • Ticket checker – TC Entry Conditions: • P wants to see a play • P has a money Results: • P saw a play • P has less money • P is happy (optional if he liked the play)	<i>Scene 1: Going to theater</i> • P PTRANS P into theater • P ATTEND eyes to ticket counter
	<i>Scene 2: Buying ticket</i> • P PTRANS P to ticket counter • P MTRANS (need a ticket) to TD • TD ATRANS ticket to P
	<i>Scene 3: Going inside hall of theater and sitting on a seat</i> • P PTRANS P into Hall of theater • TC ATTEND eyes on ticket POSS_by P • TC MTRANS (showed seat) to P • P PTRANS P to seat • P MOVES P to sitting position
	<i>Scene 4: Watching a play</i> • P ATTEND eyes on play • P MBUILD (good moments) from play
	<i>Scene5: Exiting</i> • P PTRANS P out of Hall and theater

- It must be activated based on its significance.
- If the topic is important, then the script should be opened.
- If a topic is just mentioned, then a pointer to that script could be held.
- For example, given "John enjoyed the play in theater", a script "Play in Theater" suggested above is invoked.
- All implicit questions can be answered correctly.
- Here the significance of this script is high.
 - Did John go to theater?
 - Did he buy ticket?
 - Did he have money?
- If we have a sentence like "John went to theater to pick his daughter", then invoking this script will lead to many wrong answers.
 - Here significance of the script theater is less.
- Getting significance from the story is not straightforward. However, some heuristics can be applied to get the value.

THANK YOU...